

quantum-group: A lightweight SymPy package for verifying $U_q(\mathfrak{sl}_2)$ and $GL_q(2|1)$ structures

Taha Berk Terekli

27 September 2026

Summary

quantum-group is a Python package for reproducible, symbolic matrix calculations in quantum groups. Built on SymPy (Meurer et al. 2017), it covers finite-dimensional representations of $U_q(\mathfrak{sl}_2)$ and the graded R-matrix of the quantum supergroup $GL_q(2|1)$ introduced by Çelik and Çelik (2021). Quantum groups connect representation theory with the quantum Yang–Baxter equation (QYBE), braid groups and knot invariants (Drinfeld 1987; Jimbo 1985, 1986). In super vector spaces, tensor permutations also carry parity-dependent signs.

For $U_q(\mathfrak{sl}_2)$, the package constructs the highest-weight modules V_n , checks their defining relations and generator-level Hopf identities, and computes tensor-product highest-weight vectors (Kassel 1995). It verifies the fundamental R-matrix, its coproduct compatibility, the QYBE, and braid and Hecke relations. For $GL_q(2|1)$, it constructs the published 9×9 R-matrix, verifies the graded YBE on $V^{\otimes 3}$, and checks all six R-matrix placements and all four local YBE triples on $V^{\otimes 4}$.

Statement of need

Researchers and students reading quantum-group calculations often need to check a specific matrix identity, basis convention or grading sign. A small, inspectable implementation makes these calculations reproducible without requiring a standalone computer algebra system. quantum-group provides explicit matrices and residuals that can be examined, modified and reused in Python notebooks. Its intended use is verification of concrete representation-level calculations and teaching examples.

State of the field

GAP’s QuaGroup (The GAP Group 2026; de Graaf 2024, 2001) and SageMath’s QuantumGroup interface to QuaGroup (The Sage Developers 2026) provide substantially broader functionality for quantized enveloping algebras of semisimple Lie algebras, including highest-weight modules and R-matrices. SageMath also implements crystals for quantum general linear superalgebras following Benkart et al. (2000). Thus, superalgebra computations are not absent from these systems.

Table 1: Comparison of the specific quantum-group interfaces, not all functionality of GAP or SageMath.

Capability	quantum-group	QuaGroup / SageMath QuantumGroup
Enveloping algebras	$U_q(\mathfrak{sl}_2)$	semisimple Lie types
Fundamental R-matrix examples	$4 \times 4, 9 \times 9$	general highest-weight modules
Graded YBE for the Çelik–Çelik matrix	explicit implementation and tests	not documented in the cited interfaces
Runtime	Python, SymPy, NetworkX, Matplotlib	GAP; SageMath for its interface

The contribution is a focused SymPy implementation with explicit grading conventions and executable checks for this published supergroup example. The package installs with pip from its Git repository; pytest and Jupyter are optional dependencies. It does not attempt to replace the broader algebraic algorithms of QuaGroup or SageMath.

Software design

Most checks build both sides of a matrix identity, subtract them and simplify the residual entries. Default calculations use an exact symbolic, nonzero parameter q . Rational identities hold wherever their denominators are defined; specialization at exceptional parameters needs separate analysis. A residual that does not simplify to zero is not, by itself, a proof of inequality for arbitrary SymPy expressions. Python integer parameters are converted to exact SymPy values; floating-point inputs remain approximate. The q -arithmetic helpers use Laurent-polynomial formulas to avoid removable singularities at roots of unity.

The modules follow the mathematics: `representations`, `relations`, `hopf`, `tensor`, `r_matrix` and `supergroup_gl21`. Intertwining and Yang–Baxter checks expose residual/Boolean pairs. Defining-relation checks return residuals and Booleans in structured objects; Hopf, Hecke and four-factor local YBE checks return statuses or Boolean dictionaries. Separate modules provide classical limits, root-of-unity examples, combinatorial crystal graphs and visualization.

Conventions are part of the API. With $\Delta(E) = E \otimes 1 + K \otimes E$ and $\Delta(F) = F \otimes K^{-1} + 1 \otimes F$, the compatible matrix is `R_matrix_v1_coproduct`, equal to $R_{21} = PRP$ for the historical upper-triangular `R_matrix_v1`. Here P is the ordinary flip. The latter matrix is the fundamental image of Drinfeld’s universal R-matrix, multiplied by $q^{1/2}$ to remove half powers (Drinfeld 1987, sec. 13, pp. 816–817). Tests verify $R_{21}\Delta(X) = \Delta^{\text{op}}(X)R_{21}$ for all four generators. The compatible braid PR_{21} acts by q and $-q^{-1}$ on the package’s generic V_2 and V_0 summands, respectively.

For $GL_q(2|1)$, the basis parities are $(0, 0, 1)$ and $P_s(e_i \otimes e_j) = (-1)^{p(i)p(j)}e_j \otimes e_i$. The matrix entries and the construction $R_{13} = (P_s \otimes I)R_{23}(P_s \otimes I)$ follow the unnumbered display on page 261 of Çelik and Çelik (2021). General placements use graded adjacent swaps and require an even operator. Tests compare them with an independent signed basis-action formula; replacing P_s by an ordinary flip gives a nonzero YBE residual, providing a negative control.

The suite contains 158 pytest cases. It verifies complete Clebsch–Gordan change-of-basis matrices for $V_1 \otimes V_1$, $V_2 \otimes V_2$ and $V_3 \otimes V_2$, including invertibility and all four generator actions. `MANUSCRIPT_CODE_MAPPING.md` links claims to implementations and tests. GitHub Actions is configured for Python 3.10–3.12. A reproducible benchmark script and its recorded environment document the cost of dense tensor-power calculations; these grow rapidly with tensor degree.

Research impact statement

The package originated in an undergraduate thesis at Yıldız Technical University. Its contribution is reproducible software verification of known identities, including all 729 entries of the graded YBE residual, all four-factor local triples, and disjoint-pair commutators. It does not claim new mathematical results. Finite matrix checks do not prove an abstract Hopf-algebra presentation or classify representations. The package does not implement the noncommutative Gauss decomposition or Hopf superalgebra of Çelik and Çelik (2021), a Markov trace, or a Jones polynomial. Root-of-unity examples are not a construction of the small quantum group, and the graph $B(n)$ is a combinatorial crystal model rather than a crystal-basis algorithm.

AI usage disclosure

Claude Code (Anthropic) assisted with packaging, project infrastructure, benchmarks, publication documentation, bibliography preparation and the renaming of the Hecke-check function. Codex (OpenAI, GPT-6) assisted with English translation and a subsequent mathematical and technical audit. The latter included source comparison, symbolic calculations, corrections to parameter handling and asymptotics, coproduct-compatible R-matrix APIs, regression tests, and revision of this manuscript and supporting documentation. The original core implementation and thesis predate these publication preparation changes. AI-generated calculations were checked with executable symbolic tests and against the primary sources identified in `AUDIT.md`. The author is responsible for the submitted software, mathematical claims and disclosure.

Acknowledgements

The author thanks Prof. Dr. Salih Çelik for supervising the thesis from which this package originated.

References

- Benkart, Georgia, Seok-Jin Kang, and Masaki Kashiwara. 2000. “Crystal Bases for the Quantum Superalgebra $U_q(\mathfrak{gl}(m, n))$.” *Journal of the American Mathematical Society* 13 (2): 295–331. <https://arxiv.org/abs/math/9810092>.
- Çelik, Salih, and Sultan A. Çelik. 2021. “A New Quantum Supergroup and Its Gauss Decomposition.” *Reports on Mathematical Physics* 88 (2): 259–69. [https://doi.org/10.1016/S0034-4877\(21\)00073-2](https://doi.org/10.1016/S0034-4877(21)00073-2).
- de Graaf, Willem A. 2001. “Computing with Quantized Enveloping Algebras: PBW-Type Bases, Highest-Weight Modules and R -Matrices.” *Journal of Symbolic Computation* 32 (5): 475–90.
- de Graaf, Willem A. 2024. *QuaGroup: Computations with Quantum Groups, Version 1.8.4*. Released. <https://github.com/gap-packages/quagroup>.
- Drinfeld, Vladimir G. 1987. “Quantum Groups.” *Proceedings of the International Congress of Mathematicians (Berkeley, Calif., 1986)* (Providence, RI) 1: 798–820.
- Jimbo, Michio. 1985. “A q -Difference Analogue of $U(\mathfrak{g})$ and the Yang–Baxter Equation.” *Letters in Mathematical Physics* 10: 63–69. <https://doi.org/10.1007/BF00704588>.
- Jimbo, Michio. 1986. “A q -Analogue of $U(\mathfrak{gl}(N + 1))$, Hecke Algebra, and the Yang–Baxter Equation.” *Letters in Mathematical Physics* 11 (3): 247–52. <https://doi.org/10.1007/BF00400222>.
- Kassel, Christian. 1995. *Quantum Groups*. Vol. 155. Graduate Texts in Mathematics. Springer. <https://doi.org/10.1007/978-1-4612-0783-2>.
- Meurer, Aaron, Christopher P. Smith, Mateusz Paprocki, et al. 2017. “SymPy: Symbolic Computing in Python.” *PeerJ Computer Science* 3: e103. <https://doi.org/10.7717/peerj-cs.103>.
- The GAP Group. 2026. *GAP – Groups, Algorithms, and Programming, Version 4.16.1*. Released. <https://www.gap-system.org>.
- The Sage Developers. 2026. *SageMath, the Sage Mathematics Software System (Version 10.9)*. Released. <https://doi.org/10.5281/zenodo.8042260>.